

APELLIDOS: _____ NOMBRE: _____

Bloque 1

[2p] 1. El siguiente código se ejecuta en el procesador segmentado de 5 etapas estudiado en clase: IF, ID, EX, MEM y WB. Los saltos se deciden en la etapa ID empleando predicción de salto fijo no efectivo. Hay una unidad de detección de riesgos en la etapa ID, y una unidad de anticipación en la etapa EX. La unidad de suma en punto flotante está segmentada, y requiere 3 ciclos de reloj para efectuar unaa operación de suma. Cada una de las etapas MEM y WB sólo puede gestionar una instrucción en cada ciclo.

(a) [0.4p] Muestra el diagrama multiciclo para **la primera iteración** de este código, marcando de forma explícita **las anticipaciones y los bloqueos del pipeline**.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1 addi \$s0, \$0, 4	IF	ID	EX	MEM	WB											
Loop:																
2 lwc1 \$f1, 0(\$a1)		IF	ID	EX	MEM	WB										
3 add.s \$f2, (\$f1), \$f2			IF	ID	-	A1	A2	A3	MEM	WB						
4 addi \$a1, \$a1, 4				IF	-	ID	EX	MEM	WB							
5 addi \$s0, \$s0, -1						IF	ID	-	EX	MEM	WB					
6 bne \$s0, \$0, Loop							IF	-	ID	-	-	EX	MEM	WB		
7 swc1 \$f2, 0(\$a2)									IF	-	-	ID	nop	riesgo de control		
2 lwc1 \$f1, 0(\$a1)													IF	ID	EX	MEM

(b) [0.3p] Dentro del bucle, indica dos dependencias verdaderas (de datos) y una antidependencia.

Marcadas sobre el código: verdaderas y antidependencia

(c) [0.2p] Identifica **sobre el diagrama** un riesgo estructural y un riesgo de control.

(d) [0.3p] En este código, si el procesador emplease un **predictor de saltos dinámico de 1-bit**, razona cuántas veces se predeciría el salto de forma correcta.

2 veces: fallo, acierto, acierto, fallo (3 si se considera predicción inicial efectiva)

(e) [0.3p] Partiendo del código original, si el procesador emplease salto retardado, razona sobre la posibilidad de emplear la instrucción 5 (addi \$s0, \$s0, -1) para rellenar el hueco de retardo.

Es posible, pero desde destino de salto: hay que replicarla, o bien modificar la instrucción 1 a addi \$s0, \$0, 3. Al terminar el bucle \$s0 tendrá valor -1 en lugar de 0.

(f) [0.5p] Razona cuáles de las siguientes señales de control estarían activas en cualquiera de las etapas del procesamiento durante el quinto ciclo de ejecución: MemRead, MemWrite, RegWrite, Mem2Reg, Branch, IF_flush.

WB: RegWrite (addi), MEM: MemRead (lw). El resto no se encuentran activas en el ciclo 5.

- [2p] 2. Un computador tiene una memoria principal de **1 MiB** y una memoria caché asociativa por conjuntos de **2 vías** de **512 B** con líneas de **16 bytes**, con algoritmo de reemplazo LRU. Las políticas de escritura son **post-escritura (write-back)** y **ubicar en escritura (write-allocate)**. Comenzando con la caché vacía, ejecutamos el siguiente código. La matriz A está almacenada por filas comenzando en la dirección 0x0A000, y cada elemento de tipo “int” ocupa exactamente 4 bytes. Las variables escalares se almacenan en registros de la CPU y no generan accesos a memoria.

```
int i, j, A[32][64];
for (i = 0; i < 2; i++)
    for (j = 0; j < 2; j++)
        A[j+1][i*2] = A[j][0]
```

- (a) [0.3p] Calcula la dirección de memoria ocupada por A[i][j] en función de “i” y “j”.

$$A[i][j] = 0x0A000 + 0x100i + 0x4j$$

- (b) [0.3p] Indica la partición en campos de una dirección física desde el punto de vista de la caché.

Etiqueta (12 bits), Índice (4 bits), Desplazamiento (4 bits)

- (c) [0.6p] Rellena todos los campos de la siguiente tabla de accesos a memoria: dirección, etiqueta, e índice/conjunto (en formato **hexadecimal**); indica si el acceso resulta en un acierto o un fallo caché, y en este último caso en qué tipo de fallo; e indica si el acceso provoca una escritura a memoria principal (MP) o no.

Dirección	Etiqu.	Idx	A/F	Tipo de Fallo	Escritura MP	
0A000	0A0	0	F	Forzoso		carga 0A0 en via 0
0A100	0A1	0	F	Forzoso		carga 0A1 (modificada) en via 1
0A100	0A1	0	A			
0A200	0A2	0	F	Forzoso		reemplaza 0A0 por 0A2 (mod) en via 0
0A000	0A0	0	F	Conflicto	SI	reemplaza 0A1 (mod) por 0A0 en via 1
0A108	0A1	0	F	Conflicto	SI	reemplaza 0A2 (mod) por 0A1 (mod) en via 0
0A100	0A1	0	A	Acierto		
0A208	0A2	0	F	Conflicto		reemplaza 0A0 por 0A2 (mod) en via 1

- (d) [0.3p] Si el tiempo medio de acceso a memoria fue de 43 ciclos, y el tiempo de acierto caché es de 1 ciclo, calcula el tiempo de acceso a memoria principal.

$$43 = 1 + \frac{6}{8} \times TMP \text{ ciclos} \rightarrow TMP = (43 - 1) \times \frac{8}{6} = 7 \times 8 = 56 \text{ ciclos}$$

- (e) [0.6p] Discute la veracidad de las siguientes afirmaciones:

- i. Una caché víctima reduce la tasa de fallos y, por lo tanto, también reduce el tiempo medio de acceso a memoria.

La caché víctima reduce el tiempo medio de acceso a memoria porque algunos fallos pueden resolverse en ella en lugar de acceder a MP, pero no afecta a la tasa de fallos de la caché.

- ii. En este sistema, si hubiese una caché L2 del doble de tamaño que la caché L1 y mantenido el resto de parámetros de la caché idénticos, ningún fallo provocaría una escritura a memoria principal.

Es verdadero: al duplicar el tamaño, el índice en L2 tendrá 1 bit más que en L1 y los accesos con etiqueta A01 se mapean a un conjunto diferente, por lo que los fallos de conflicto se resolverán en esa caché L2.

- iii. Si esta caché tuviese una política de escritura directa (write-through), no habría fallos de conflicto.

Falso. Sería cierto para una caché que no ubique en escritura, pero la política de escritura directa no necesariamente fuerza la política de ubicación.

APELLIDOS: _____ NOMBRE: _____

Bloque 2

[0.75p] 3. Un programa consta de 10^9 instrucciones y tarda 1.4 segundos en ejecutarse. Involucra únicamente dos subsistemas en la ejecución: CPU y E/S. El 30% de las instrucciones son sumas de valores de doble precisión, que tardan cada una exactamente 4 ciclos en ejecutarse cada una. El 10% son instrucciones de E/S que tardan de media 10 ciclos cada una. El resto de operaciones son de tipo entero y se ejecutan en 1 ciclos de reloj.

(a) Calcula el CPI.

$$\frac{10^9 \times (0.3 \times 4 + 0.1 \times 10 + 0.6 \times 1) \text{ ciclos}}{10^9 \text{ instrucciones}} = 2.8 \text{ ciclos/instr}$$

(b) Calcula la métrica FLOPS para esta ejecución.

$$\frac{0.3 \times 10^9 \text{ flops}}{1.4 \text{ s}} = 2.14 \times 10^8 = 214 \text{ MFLOPS}$$

(c) Calcula la frecuencia de reloj.

$$f_{cpu} = \frac{2.8 \times 10^9 \text{ ciclos}}{1.4 \text{ s}} = 2 \times 10^9 \text{ ciclos/s} = 2 \text{ GHz}$$

(d) En una segunda versión de esta máquina se consigue una aceleración de 2.5 en las operaciones de entrada salida. El resto de operaciones no se ven afectadas. ¿Cuánto más rápido se ejecuta el programa en esta máquina con respecto a la anterior?

$$A = \frac{10^8 \times (3 \times 4 + 1 \times 10 + 6 \times 1) \times \cancel{T_{ciclo}}}{10^8 \times (3 \times 4 + 1 \times 10/2.5 + 6 \times 1) \times \cancel{T_{ciclo}}} = \frac{28}{22} = 1.27$$

o bien

$$F_m = \frac{1 \times 10}{3 \times 4 + 1 \times 10 + 6 \times 1} = 0.357$$

$$A_m = 2.5$$

$$A = \frac{1}{1 - 0.357 + \frac{0.357}{2.5}} = 1.27$$

(e) Este programa forma parte de un benchmark SPEC. Al ejecutarlo 3 veces se han medido tiempos de 1.3, 1.1 y 1.6 segundos. La máquina de referencia para ese benchmark ha tardado 24 segundos en ejecutarlo. Calcula el ratio SPEC individual para este programa.

Para el cálculo del ratio SPEC se utiliza el valor mediano: 1.3 segundos.

$$\frac{24 \text{ s}}{1.3 \text{ s}} = 18.46$$

[2p] 4. Un computador con 1 GiB (2^{30} bytes) de memoria principal utiliza memoria virtual paginada con traducción en 2 niveles. El tamaño de página es de 4 KiB (2^{12} bytes). Cada tabla de páginas involucrada en el proceso de traducción ocupa exactamente una página física, y cada entrada comienza con 1 bit de residencia, seguido de 1 bit de modificación, 12 bits adicionales de protección y control, y el número de página física.

(a) Determina el tamaño del espacio virtual.

Una DF, de 30 bits, se divide en 12 bits de desplazamiento y 18 bits de página física. Cada entrada de la tabla de páginas ocupa $1 + 1 + 12 + 18 = 32$ bits = 4 bytes. Una tabla de páginas, que ocupa 2^{12} bytes, contiene $2^{12}/4 = 2^{10}$ entradas. La DV se dividirá en 10 bits PV1, 10 bits PV2 y 12 bits Despl. Por tanto, la DV tiene 32 bits y el espacio virtual es de 2^{32} bytes = 4 GiB.

(b) En la traducción de la dirección virtual 0x0116453C, la entrada correspondiente de la tabla de páginas de último nivel (la última tabla consultada antes de terminar el proceso) contiene el valor 0x90431F45. Razona si se puede hacer una traducción válida y calcula la dirección física resultante.

El desplazamiento de la DV es 0x53C. El bit de residencia de la entrada es 1 (0x9 = 1001), por lo que la traducción se puede realizar. El número de PF son los últimos 18 bits de la entrada: 0x31F45. La DF resultante es 0x31F4553C.

(c) La siguiente tabla muestra el contenido de la TLB de un proceso. Indica una dirección virtual que pueda ser traducida a través de esta TLB, y cuál sería la dirección física correspondiente.

PV	PF
0x44510	0x1F34C
0xD178C	0x2ABD0
0x44511	0x0A1B3
0x3A022	0x1BD42

Si escogemos por ejemplo la primera entrada de la TLB y utilizamos un desplazamiento arbitrario, como por ejemplo 0, la DV 0x44510000 se traduciría en la DF 0x1F34C000.

(d) Describe brevemente una utilidad de la memoria virtual

Ejemplo de respuestas posibles: (i) ejecución de programas más grandes que la memoria física, (ii) aislamiento de procesos, (iii) ejecución de múltiples programas que en conjunto ocupan más que la memoria física disponible, (iv) flexibilidad en la asignación de memoria

(e) Indica una ventaja y un inconveniente de una caché virtual con respecto a una caché física en la gestión de la memoria virtual

Ejemplo de respuesta: no es necesario traducir direcciones para acceder a una caché virtual porque las etiquetas e índices se obtienen de la dirección virtual, pero es necesario implementar un mecanismo para garantizar la seguridad y el aislamiento entre procesos, como por ejemplo incluir el PID del proceso en el directorio caché o reemplazar todo el contenido de la caché en cada cambio de contexto.

APELLIDOS: _____ NOMBRE: _____

Bloque 3

[0.5p] 5. Completa correctamente las siguientes afirmaciones:

- (a) Suponiendo discos de 1 TiB y una capacidad neta de almacenamiento de 8 TiB, un **RAID 6** tendrá una capacidad de almacenamiento total de **10 TiB**.
- (b) Algunas de las ventajas de los discos de estado sólido **SSD** frente a los discos duros magnéticos **HDD** son ser **más rápidos** y más silenciosos.
- (c) Los discos duros (**HDD**) y las unidades de estado sólido (**SSD**) forman parte del nivel de memoria **secundaria** de un computador.
- (d) Los **RAID 4** y **5** son similares, pero el **RAID 5** no tiene el problema de cuello de botella en el acceso a la información de paridad.
- (e) Cuando queremos implementar un **RAID**, si utilizamos un único modelo de disco de un fabricante, es aconsejable que sean de **distinta serie de fabricación** para aumentar la fiabilidad.

[0.75p] 6. Tenemos un sistema con memoria virtual de páginas de 4 KiB que conecta la memoria principal con el almacenamiento secundario mediante un bus síncrono de 64 bits a 50 MHz.

Las transferencias de las páginas están gestionadas por una DMA a través de este bus en transacciones de 128 palabras (8 bytes/palabra), necesitando 300 ns para la lectura de la primera palabra y 150 ns para el resto.

Tanto el envío de 64 bits a través del bus como el de la dirección al controlador del almacenamiento secundario requieren un ciclo. Las transferencias de datos leídos más recientemente pueden solaparse con la lectura de los siguientes. Entre transacciones de bus es necesario esperar 2 ciclos de reloj de bus.

Calcula la latencia y el ancho de banda para la lectura de una página.

$$T_{\text{ciclo_bus}} = \frac{1}{f} = \frac{1}{50 \text{ MHz}} = \frac{1}{5} 10^{-7} \text{ s} = 20 \text{ ns}$$

El sistema soporta transacciones de 128 palabras. Cada página son 4 KiB, que corresponden a $4 \text{ KiB} / (8 \text{ B/p}) = 2^9$ palabras. Por tanto, para transferir una página, se requieren: $2^9 \text{ p} / (128 \text{ p/transacción}) = 4$ transacciones.

Para leer las 128 palabras de 1 transacción necesitamos:

- 1 ciclo para enviar la dirección: 20 ns.
- Un tiempo de acceso a memoria de 300 ns para leer la primera palabra.
- 150 ns para acceder a cada una de las siguientes palabras que se solapa con el envío de la anterior palabra leída (20 ns).
 - Como cada transacción es de 128 palabras necesitamos 127 lecturas adicionales (el tiempo de lectura es superior al tiempo de envío).
- 1 ciclo de envío para la última palabra.
- Al finalizar una transacción hay que esperar 2 ciclos de reloj para empezar con la siguiente: 40 ns entre transacciones.

Por tanto, para la lectura de las 128 palabras de 1 transacción necesitamos:

$$200 + 300 + 127 \times 150 + 20 = 19390 \text{ ns/transacción.}$$

La latencia para la lectura de una página será:

$$4 \text{ transacciones} \times 19390 \text{ ns/transacción} + 3 \text{ esperas} \times 40 \text{ ns/espera} = \boxed{77680 \text{ ns} \simeq 77.7 \mu\text{s}}$$

$$\text{Y el ancho de banda será: } 4 \text{ KiB} / (77680 \times 10^{-9} \text{ s}) \simeq \boxed{50.3 \text{ MiB/s} \simeq 52.7 \text{ MB/s}}$$